

Petrophysical Software Engine

Master Prompt Guide for Claude 3.5 Sonnet (Cursor IDE)

This document contains the exact, sequence-optimized prompts required to build a high-performance petrophysical desktop application capable of processing 2000+ wells. The stack utilizes **Tauri (Rust)**, **DuckDB**, and **WebGPU**.

Execution Strategy: Do not paste all prompts at once. Feed them to Claude in Cursor sequentially. Wait for Claude to implement, review the code, verify the application compiles (using `cargo check` or `npm run dev`), and then proceed to the next step.

Phase 0: IDE Configuration

Before coding, you must configure the AI's internal logic. Create a file named `.cursorrules` in your project's root directory and paste the following text. You do not need to prompt Claude for this; simply save the file.

You are an expert petrophysicist, systems architect, and high-performance Rust/TypeScript developer.

The project is a desktop application built on Tauri, using DuckDB for embedded columnar data storage, Rust for multi-threaded analytics, and TypeScript/WebGPU for hardware-accelerated rendering.

CRITICAL IMPLEMENTATION RULES:

1. **Data Storage:** Always optimize DuckDB queries for columnar reading. Use compressed binary blobs or LIST types for array logs (NMR, waveforms).
2. **Missing Values:** Never use Option<f32> for continuous logs. Missing data must strictly be represented as f32::NAN to maintain clean matrix arithmetic performance.
3. **Serialization & IPC:** Avoid passing raw data arrays over Tauri's IPC bridge as JSON strings. Convert Vec<f32> matrices directly into raw byte vectors (Vec<u8>) using the `bytemuck` crate, and cast them back to Float32Array on the frontend.
4. **Concurrency:** Use `rayon` for fast cell-based parallel operations across CPU cores and `tokio` for background asynchronous scheduling.
5. **Code Delivery:** Keep code implementations concise, modular, and focused on production-grade execution speed. Do not generate extensive unit test blocks unless explicitly requested.

Phase 1: Database Ingestion & Schema

This prompt establishes the highly optimized DuckDB database, separating standard analytical curves from high-resolution thin-bed data.

I am building the backend for a high-performance petrophysics software using Tauri (Rust) and DuckDB. The goal is to load and query 2000+ wells instantly. I need you to write the Rust module for the initial DuckDB database setup and ingestion schema.

Please provide the `Cargo.toml` dependencies and the `main.rs` (or `db.rs`) implementation meeting these strict requirements:

1. Initialize DuckDB: Create an embedded DuckDB connection file (e.g., `project.duckdb`).
2. Schema Creation: Write the SQL execution block to create the following tables:
 - `wells`: `well_id` (UUID), `well_name`, `field_name`, `td`, `kb`.
 - `standard_curves`: `well_id`, `depth` (FLOAT), `GR`, `RES_DEEP`, `NPHI`, `RHOB`. (Make `well_id` and `depth` a composite primary key).
 - `high_res_curves`: `well_id`, `depth`, `micro_res`, `image_pad`. (Strictly separated for thin bed evaluation).
 - `lqr_parameters`: `well_id`, `depth`, `clay_volume`, `capillary_pressure`, `microporosity`. (Designed for Low Quality Reservoir and LRLC analysis).
 - `array_logs`: `well_id`, `depth`, `nmr_t2_distribution` (FLOAT[] array type).
3. Rust Structs: Create the corresponding Rust `struct` definitions (using `serde`) that represent a standard LAS curve row so we can deserialize incoming data.
4. Batch Ingestion Function: Write a Rust function `insert_standard_curves(well_id, depths: Vec<f32>, gr: Vec<f32>)` that uses DuckDB's Appender or prepared statements to bulk-insert 10,000+ rows in a single transaction efficiently.

Focus heavily on Rust performance, using `Result` for robust error handling, and ensure the SQL syntax is perfectly optimized for DuckDB's columnar nature.

Phase 2: High-Speed File Parsers (LAS 2.0 & Geolog CSV)

This prompt instructs Claude to build memory-safe data parsers to feed the DuckDB database.

I need to build two high-performance file parsers in my Rust Tauri backend to feed the DuckDB database we just created. I will be parsing standard LAS 2.0 files and massive CSV exports.

Please provide the Rust implementation using the `csv` crate and `std::io::BufReader`.

1. CSV Parser (`parse_geolog_csv`):

- Write a function that reads a CSV file containing well log data.
- Use the `csv` crate and `serde` to deserialize the rows into a struct: `LogDataRow { depth: f32, gr: Option<f32>, res: Option<f32>, rhob: Option<f32> }`.
- Map missing values directly to `f32::NAN`.
- The function should return a collection of these columns as `Vec<f32>` arrays, ready to be handed to the DuckDB Appender.

2. LAS 2.0 Parser (`parse_las_2`):

- Write a custom parser using `BufReader::new(File::open(path)?)` to stream the file line-by-line (do not load the whole file into RAM).
- Implement a simple state machine that looks for the `~C` (Curve) block to map column indices, and the `~A` (ASCII) block to start reading data.
- Parse the ASCII lines into floating-point numbers. If a value matches a standard null value (like `-999.25` or `-9999.0`), strictly map it to `f32::NAN`.

3. Multithreading:

- Provide a brief example using `rayon` to show how I can pass a directory of 500 LAS files to the `parse_las_2` function and process them concurrently across all CPU threads.

Phase 3: The Zero-Copy Binary IPC Bridge & Decimation

This prompt is critical for UI performance. It prevents the app from freezing when sending millions of data points to the frontend.

I need to pass massive arrays of well log data from my Rust/DuckDB backend to my Tauri frontend without freezing the UI. Standard JSON serialization is too slow.

Please write the Rust and TypeScript code for a zero-copy (or low-copy) binary IPC bridge.

1. Data Decimation (Rust):

- Write a high-speed `min_max_decimate` function that takes `depths: &[f32]`, `values: &[f32]`, and a `target_pixel_height: usize`.
- It must downsample the data using the Largest Triangle Three Buckets (LTTB) or a Min/Max visual bucketing algorithm to perfectly preserve spikes, reducing the array to match the pixel height. Ignore NaN values gracefully.

2. The Tauri Command (`get_curve_binary`):

- Write a `#[tauri::command]` that fetches a mocked `Vec<f32>` of decimated data.
- Convert this `Vec<f32>` into a raw byte vector (`Vec<u8>`) using the `bytemuck` crate. Return this byte array directly to the Tauri frontend.

3. The TypeScript Frontend Receiver:

- Write a frontend function using `@tauri-apps/api/core` `invoke`.
- Fetch the byte array and cast that raw `Uint8Array` directly into a `Float32Array` in JavaScript so it is immediately ready for a WebGPU buffer.

Phase 4: WebGPU Frontend Rendering

This prompts Claude to build the hardware-accelerated canvas that draws the logs using the graphics card.

I now have a TypeScript `Float32Array` containing my downsampled well log data arriving in under 2 milliseconds from the Rust backend.

I need you to build a high-performance frontend log plot viewer using TypeScript and an HTML5 Canvas optimized for WebGL2 or WebGPU.

Requirements:

1. Write a `LogCanvasRenderer` class that accepts an `HTMLCanvasElement`.
2. Connect it to the Tauri IPC bridge (`get_curve_binary`) to ingest the binary curve array based on the canvas's `clientHeight`.
3. Implement the WebGL2/WebGPU buffer allocation to load the `Float32Array` directly into GPU memory.
4. Write a simple Vertex and Fragment shader to render the log curve as a continuous line (handling NaN values by breaking the line).
5. Ensure the rendering loop is capable of hitting 60 FPS, with hooks for pan and zoom events that will later re-trigger the backend decimation command.

Phase 5: Multi-Processor Petrophysical Engine

This prompt sets up the multithreaded calculation framework for deterministic evaluations (e.g., across 55 fields).

Now we need to implement the actual petrophysical calculation engine in the Rust backend. I need to run deterministic models (like Vshale, Porosity, Water Saturation) across 2,000+ wells simultaneously.

Requirements:

1. Deterministic Math Engine (`rayon`):

- Write a `[tauri::command]` named `run\_multi\_well\_evaluation`.
- Accept a list of `well\_ids` and a struct `EvaluationParams { vsh\_cutoff: f32, a: f32, m: f32, n: f32, rw: f32 }`.
- Use `rayon`'s `par\_iter()` to distribute the calculation of these equations concurrently across all available physical CPU cores.
- Ensure all math correctly propagates `f32::NAN` values if the input curves have missing data.

2. Asynchronous Inversion Hooks (`tokio`):

- Create a secondary `tokio` async task queue template. Show how I would dispatch a long-running, iterative multi-mineral inversion (like a 3-minute LRLC stochastic solver) to a background thread so the Tauri UI remains perfectly responsive while it computes.